

Trade Offs between TRM/LDT Hybrids

Patrick Dugan

June 19, 2026

Abstract

We study a small hybrid reasoning architecture that combines a Tiny Recursive Model (TRM) style proposal mechanism with an explicit Lattice Deduction Transformer (LDT) style state. The central design is a typed membrane: the latent side proposes refinements, while the explicit lattice side certifies, rejects, or soft-stores them according to provenance. The goal is not to claim that a lattice guarantees correct abstention in all settings. The narrower claim is that an explicit lattice makes abstention, bottom, monotonicity, and provenance inspectable in domains where the relevant mechanics are checkable.

We evaluate this design on five lightweight benchmarks: Sudoku, ARC-1, ARC-2, environment-pointer routing, and a coupled storyworld playing task. Across the saved runs, hybrid matches the best aggregate score in every task family. It improves proposal efficiency in Sudoku and aggregate ARC-2, repairs unsafe TRM actions in storyworld play, and avoids hard-eliminating routes when LDT evidence is only lexical and therefore not environment-sound. The main negative result is that hybrid is not automatically superior: in routing it only ties TRM when LDT candidate sets are treated as soft evidence, and in two ARC-2 instances its heuristic proposal order uses more proposals than raw TRM.

1 Motivation

The motivating question is practical:

Can persistent latent recurrence improve explicit lattice deduction without destroying the interpretability and soundness benefits of an explicit abstract state?

We separate two roles. TRM is private recurrent or heuristic workspace that proposes actions, refinements, routes, or candidate rules. LDT is explicit public lattice state that supports monotone refinement, bottom detection, serialization, and typed provenance. The hybrid architecture is useful only if these roles remain separated. If latent proposals can directly mutate durable state, the system loses the main benefit of explicit deduction. If the lattice is the only mechanism, the system struggles when local propagation is insufficient and search or heuristic proposal is needed.

The terminology is motivated by recent compact recurrent reasoners, especially HRM and TRM [11, 7], and by LDT’s explicit lattice-state view [5]. The control-flow question is also adjacent to neurosymbolic architecture taxonomies and systems that combine neural proposal with symbolic, logical, or executable structure [8, 4, 9, 6, 12]. Our contribution is narrower than that literature: we study when a typed membrane should convert a proposal into a durable lattice update, keep it as soft telemetry, or reject it.

This paper intentionally excludes VPD. The benchmark argument is about hybrid reasoning behavior, typed state updates, and empirical task performance.

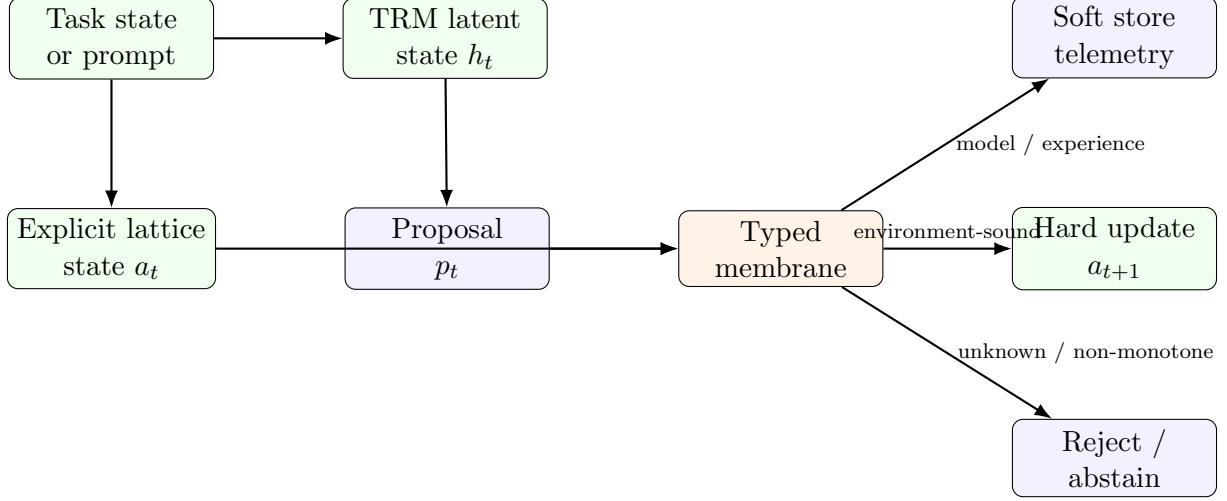


Figure 1: Hybrid LDT/TRM control loop. The TRM side proposes; the membrane decides whether the proposal can become a durable lattice update.

2 Architecture

At step t , the hybrid keeps two states:

a_t explicit abstract state, h_t private latent or heuristic state.

The public state a_t is serializable and checkable. In the implementation, it is often a candidate-set lattice, but the same interface covers intervals, symbolic constraints, reachable regions, possible rules, and possible environment IDs. The private state h_t ranks proposals and tracks search context. It does not directly mutate a_t .

3 Lattice Formulation

For the candidate-state implementation, following the lattice view used in abstract interpretation [3], let slots be indexed by $i \in I$. Each slot has finite universe U_i . A lattice state is

$$a = \{C_i \subseteq U_i\}_{i \in I}.$$

The top state is

$$\top = \{U_i\}_{i \in I},$$

and bottom is

$$\perp(a) \iff \exists i \in I : C_i = \emptyset.$$

Meet is pointwise intersection:

$$(a \wedge b)_i = C_i^a \cap C_i^b.$$

A proposal p_t contains a proposed state $\hat{a}_{t+1}$. It is monotone only if

$$\forall i \in I, \quad \hat{C}_{i,t+1} \subseteq C_{i,t}.$$

The implementation rejects non-monotone proposals before applying meet:

$$\text{reject}(p_t) \quad \text{if} \quad \exists i : \hat{C}_{i,t+1} \not\subseteq C_{i,t}.$$

This matters because otherwise a widening proposal could be hidden by a subsequent meet and appear harmless.

4 Typed Soundness and the Membrane

Each proposal carries provenance:

- ENVIRONMENT-SOUND: derived from trusted environment mechanics.
- MODEL-SOUND: derived from a model of another agent or learned predictor.
- EXPERIENCE-SOUND: derived from replay or discovered successes.
- unknown: no usable provenance.

The default membrane policy is:

$$\text{hard}(p) = \begin{cases} 1 & \text{if provenance}(p) = \text{ENVIRONMENT-SOUND}, \\ 0 & \text{otherwise.} \end{cases}$$

The durable transition is

$$a_{t+1} = \begin{cases} a_t \wedge \hat{a}_{t+1} & \text{if } \text{hard}(p_t) \wedge \hat{a}_{t+1} \sqsubseteq a_t \wedge \neg \perp(a_t \wedge \hat{a}_{t+1}), \\ a_t & \text{otherwise.} \end{cases}$$

Bottom is allowed only in explicit abstain or conflict mode:

$$\perp(a_t \wedge \hat{a}_{t+1}) \Rightarrow \text{accept only if mode} = \text{ABSTAIN}.$$

5 Control Dynamics and Alternative Architectures

The typed membrane is one point in a larger design space. We compare it with two implementable alternatives in order to separate the value of hybridization from the risks introduced by the control policy.

Let q_θ denote the TRM proposal or scoring mechanism. In the typed membrane,

$$p_t = q_\theta(h_t, a_t, x_t), \quad p_t = (\hat{a}_{t+1}, \tau_t, m_t),$$

and durable state changes only through the membrane:

$$a_{t+1} = \begin{cases} a_t \wedge \hat{a}_{t+1} & \tau_t = \text{ENVIRONMENT-SOUND}, \hat{a}_{t+1} \sqsubseteq a_t, \neg \perp(a_t \wedge \hat{a}_{t+1}), \\ a_t & \text{otherwise.} \end{cases}$$

The first alternative is a hard-gated cascade. The LDT state is treated as a mandatory candidate filter:

$$C_t = \text{LDT}(x_t), \quad y_t = \arg \max_{y \in C_t} q_\theta(y \mid x_t).$$

This is attractive when C_t is environment-sound, but unsafe when C_t is lexical or learned. The routing ablation below is exactly this failure mode.

The second alternative is confidence arbitration. Let $s_1(x_t)$ and $s_2(x_t)$ be the top two TRM scores and let

$$\Delta_t = s_1(x_t) - s_2(x_t).$$

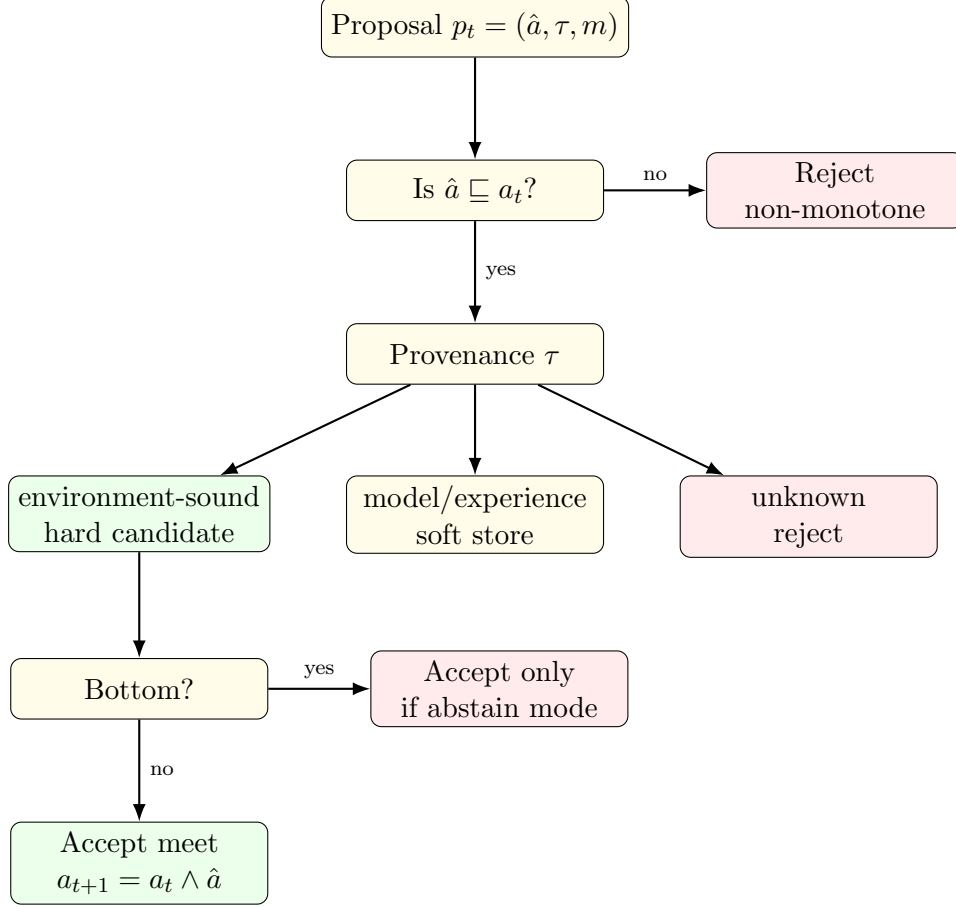


Figure 2: Default membrane policy. Only environment-sound monotone refinements become hard updates. Model-sound and experience-sound proposals can be retained as soft telemetry.

Then

$$y_t = \begin{cases} \arg \max_y q_\theta(y \mid x_t) & \Delta_t \geq \gamma, \\ \arg \max_{y \in C_t} q_\theta(y \mid x_t) & \Delta_t < \gamma. \end{cases}$$

The threshold γ is selected on the training split from a small deterministic grid. On the saved routing run the selected value is $\gamma = 0$, meaning that any positive use of noisy LDT routing candidates reduces training accuracy; the arbitration architecture therefore degenerates to TRM on this slice.

6 Practical Hybrid Dynamics

The hybrid does not play the same role in every domain. Figure 5 summarizes the main patterns.

6.1 Sudoku

LDT performs naked-single propagation. TRM performs heuristic minimum-remaining-values search. Hybrid lets TRM propose a branch and then applies LDT propagation after the proposal. TRM is useful for escaping propagation plateaus; LDT is useful immediately after proposals because it collapses implied consequences.

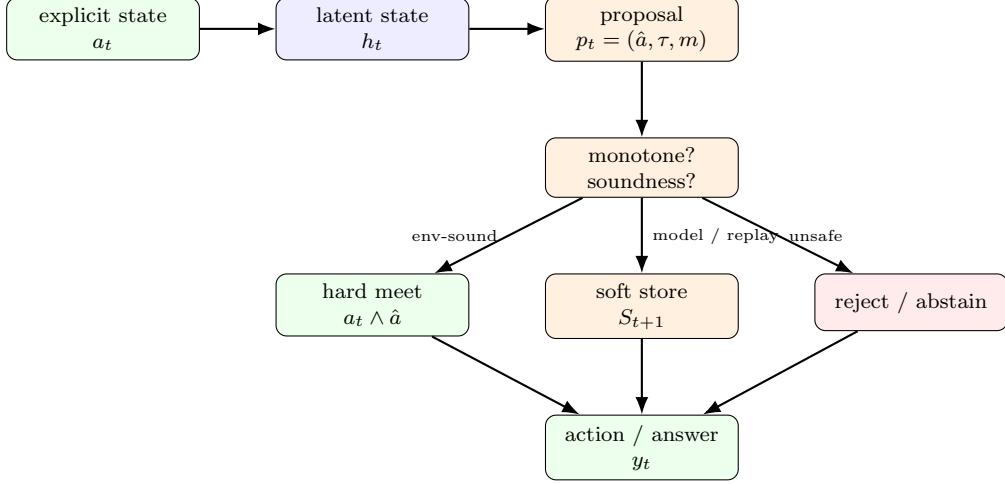


Figure 3: Control dynamic for the typed membrane. TRM proposes; the membrane decides whether the proposal becomes a hard meet, soft telemetry, or a rejection.

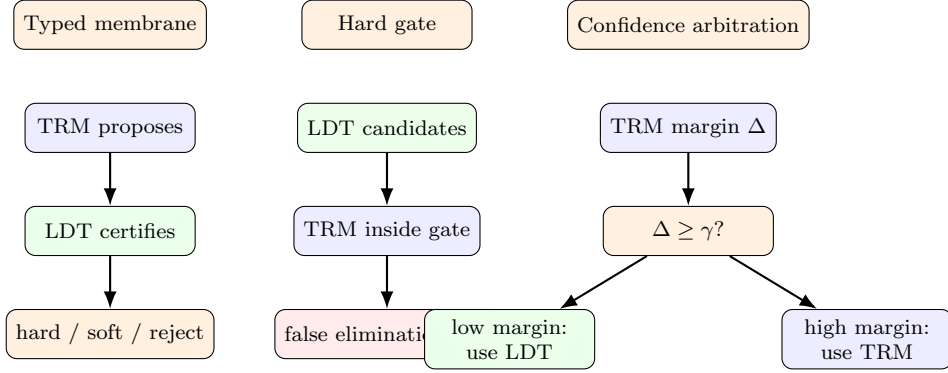


Figure 4: Three benchmarked hybrid control policies: typed membrane, hard-gated cascade, and confidence arbitration.

6.2 ARC-1 and ARC-2

ARC-1 is a single-rule grid transformation benchmark inspired by the Abstraction and Reasoning Corpus framing [2]. ARC-2 uses ordered two-rule compositions. LDT derives broad rule-family constraints from input/output invariants. TRM searches primitive rules or ordered pairs. Hybrid restricts proposals by LDT families and certifies them against training examples:

$$\text{accept}(\hat{r}) \iff \forall (x_j, y_j) \in D_{\text{train}} : \hat{r}(x_j) = y_j.$$

For ARC-2, $\hat{r} = r_b \circ r_a$ and the same certification applies.

6.3 Environment-Pointer Routing

Routing maps a prompt to an environment ID. The Tesseract router reference trains a TF-IDF plus MLP router [1]. The dependency-light benchmark here uses a lexical TRM analogue. LDT builds token-derived candidate sets:

$$C_t = \bigcap_{w \in \text{tokens}(x)} E_w,$$

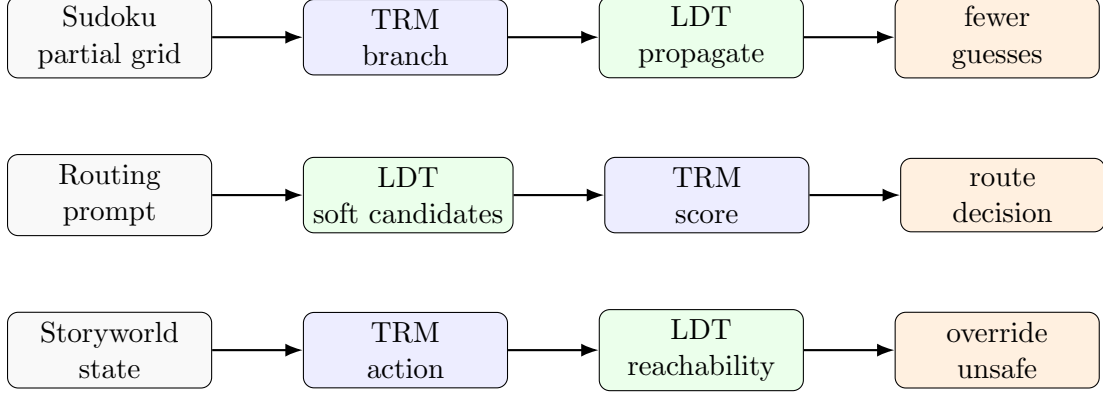


Figure 5: Three practical hybrid patterns: branch then propagate, soft route telemetry, and action proposal plus reachability override.

where E_w is the set of environments associated with token w . This evidence is not environment-sound. It is model- or experience-derived. Therefore the final hybrid design uses it as soft guidance only:

$$\text{route}_{\text{hybrid}}(x) = \begin{cases} \arg \max_{e \in C_t} \text{TRM}(e | x) & \text{if TRM top route is in } C_t, \\ \arg \max_e \text{TRM}(e | x) & \text{otherwise.} \end{cases}$$

6.4 Storyworld Playing

The storyworld has exact transitions and a modeled rival policy. LDT uses finite-horizon reachability. TRM uses a greedy local-deficit heuristic. Hybrid lets TRM propose an action, then checks whether the action preserves modeled reachability:

$$\text{safe}(s, a, H) = \text{Reachable}_{\text{model}}(T(s, a, \pi_{\text{rival}}(s)), H - 1).$$

Action selection is:

$$a_t = \begin{cases} a_t^{\text{TRM}} & \text{if } \text{safe}(s_t, a_t^{\text{TRM}}, H - t), \\ a_t^{\text{LDT}} & \text{if } a_t^{\text{LDT}} \text{ exists,} \\ a_t^{\text{TRM}} & \text{otherwise.} \end{cases}$$

This is also where the richer storyworld intuition enters the paper. Secret endings and morality optimization are not the same control problem. A secret ending is a typed gate: the system either preserves reachability to a specific latent ending predicate or it does not. A morality objective is a soft preference surface: it can reward trust, evidence, low heat, and scene progress without being an environment-sound proof obligation. This mirrors the difference between worlds such as medical triage or bioethics councils, where some constraints are hard safety gates while others are soft value tradeoffs.

7 Results

The aggregate pattern is:

- TRM helps when proposal, search, or lexical classification is needed.
- LDT helps when mechanics are explicit and checkable.

Benchmark	LDT	TRM	Hybrid	Best
Sudoku	0.333	1.000	1.000	TRM, Hybrid
ARC-1	1.000	1.000	1.000	All
ARC-2	1.000	1.000	1.000	All
Routing	0.474	0.815	0.815	TRM, Hybrid
Storyworld	1.000	0.172	1.000	LDT, Hybrid

Table 1: Saved benchmark scores. Scores are solve rate or routing accuracy depending on the task family.

Routing variant	Accuracy	Correct	Abstained	Avg. candidates
TRM	0.815	141/173	0	0.00
Soft hybrid	0.815	141/173	0	2.09
Hard-filter hybrid	0.584	101/173	0	2.09

Table 2: Routing ablation. Token-lattice candidate sets help as telemetry but are too noisy for hard elimination.

- Hybrid helps when proposal can be followed by checkable pruning, certification, or reachability checks.
- Hybrid does not automatically improve learned routing if the LDT side has only soft lexical evidence.

In Sudoku, hybrid matches TRM’s solve rate while reducing guesses from 31 to 6. In ARC-1, hybrid matches the best score and uses fewer steps than LDT and fewer proposals than TRM. In ARC-2, hybrid matches the best score and reduces aggregate proposals versus TRM, 32 versus 38, but is inferior to TRM on two individual task proposal counts because the current proposal ordering is heuristic. In routing, hybrid matches TRM accuracy only after treating LDT candidate sets as soft guidance; forcing LDT candidates as hard filters drops accuracy to 0.584. In storyworld play, hybrid matches LDT success and slightly reduces average steps, using 124 overrides to repair unsafe TRM proposals.

Transcript slice. One secret-ending run starts at (trust = 3, evidence = 1, heat = 4, scene = 1). Confidence arbitration sees high local margins and repeatedly selects **defuse: defuse, defuse, defuse, defuse**. It ends at (3, 1, 0, 5) and fails because evidence never reaches the secret gate. The typed membrane instead accepts the need to preserve reachability and follows **defuse, wait, wait, investigate, defuse, investigate**, ending at (1, 3, 2, 5) and satisfying the secret-ending predicate.

8 Design Decisions and Alternatives

Hard versus soft application. We hard-apply only environment-sound refinements by default. An alternative is to allow model-sound or experience-sound hard updates. This would make routing and replay-derived pruning more aggressive, but it risks false eliminations. The routing benchmark demonstrates the risk: hard LDT filtering from token evidence underperformed TRM, with the hard-filter hybrid ablation at 0.584 versus 0.815 for TRM and soft hybrid.

Architecture	Accuracy	Correct	Control policy
Typed membrane	0.815	141/173	soft telemetry unless sound
Hard gate	0.584	101/173	LDT candidates hard-filter TRM
Confidence arbitration	0.815	141/173	train-selected $\gamma = 0$

Table 3: Implemented routing architecture variants. Hard gating is inferior; confidence arbitration selects the TRM-only boundary on this data.

Regime	Preferred policy	Reason
Environment-sound gate	Typed membrane or hard gate	Proof-like state should dominate confidence
Soft preference objective	Confidence arbitration	Local score can optimize value tradeoffs
Noisy lexical evidence	Typed membrane	Keep learned candidates soft
Checkable search space	Typed membrane	Propose with TRM, certify with LDT

Table 4: Policy-selection rule of thumb from the saved benchmarks.

Reject non-monotone proposals before meet. We reject proposals that widen candidate sets before applying meet. An alternative is to apply meet regardless and accept the result if the final state does not widen. This hides proposal defects. Rejecting early gives a cleaner training signal.

Bottom requires explicit abstain. Bottom-producing proposals are rejected unless the proposal is in explicit abstain mode. An alternative is to treat bottom as an ordinary conflict result. We avoid this because bottom is semantically important and should become a visible conflict or abstention frame.

LDT as certifier, planner, or telemetry. We use LDT differently by domain: propagator in Sudoku, rule certifier in ARC, soft candidate telemetry in routing, and reachability planner in storyworld play. Forcing a single LDT role would be architecturally cleaner but less honest because checkability differs across domains.

Hybrid override policy. In storyworld play, the hybrid overrides TRM when its proposed action loses modeled reachability and a certified alternative exists. Always routing through LDT first would collapse the hybrid into LDT. Letting TRM override LDT for speed would lose the safety benefit on dynamic tasks.

Proposal ordering. Current ARC proposal ordering is simple and hand-coded. The ARC-2 efficiency regressions indicate that a learned TRM proposal ranker should be trained while retaining LDT certification.

9 Limitations

The benchmarks are small and synthetic. They test contracts, not scale. The TRM implementations in this repository are dependency-light analogues, not full recurrent neural models [11, 7]. The Tesseract router reference uses a TF-IDF plus MLP architecture, and the local routing benchmark mirrors the classification objective without importing its full PyTorch stack [1]. The LDT side is symbolic and domain-specific rather than a learned lattice transformer [5]. A learned LDT head is

Scenario	Policy	Success rate	Avg. score
Secret ending	Typed membrane	1.000	1.00
Secret ending	Confidence arbitration	0.938	0.94
Moral optimization	Typed membrane	1.000	9.95
Moral optimization	Confidence arbitration	1.000	11.06

Table 5: Storyworld confidence/type split. Typed reachability wins on hard secret gates; confidence arbitration is useful on a soft morality score.

future work. The hybrid has no aggregate score regression in the saved benchmarks, but it does have efficiency regressions in ARC-2 subcases and no routing accuracy gain over TRM yet.

10 Next Experiments

1. Train ARC-2 proposal ordering and compare proposal counts against the static hybrid.
2. Replace the routing confidence grid with a richer calibration signal; the current trained threshold degenerates to TRM on this slice.
3. Convert membrane decisions into SFT records and train a small learned membrane policy.
4. Run the INTELLECT-3 `logic-env` under WSL or Linux [10]. The Windows smoke command currently fails before environment loading due to a Unix-only `fcntl` import in `prime_tunnel`.
5. Scale storyworld play to larger GPTStoryworld or SweepWeave tasks after this finite-state harness is stable.

11 Conclusion

The core result is not that hybrid reasoning is always better. The result is more specific: a typed membrane lets latent proposal and explicit deduction cooperate without conflating their evidence types. TRM-style proposal is effective for search and routing. LDT-style explicit state is effective when mechanics are checkable. Hybrid works best when proposals can be followed by monotone refinement, certification, or reachability checks. When the LDT side has only learned or lexical evidence, it should remain soft.

References

- [1] Tesseract router trm training scaffold.
- [2] François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.
- [3] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. *Proceedings of the ACM Symposium on Principles of Programming Languages*, 1977.
- [4] Artur d’Avila Garcez and Luis C. Lamb. Neurosymbolic ai: The 3rd wave, 2020.

- [5] Liam Davis, Leopold Haller, Alberto Alfarano, and Mark Santolucito. Lattice deduction transformers, 2026.
- [6] Honghua Dong, Jiayuan Mao, Tian Lin, Chong Wang, Lihong Li, and Denny Zhou. Neural logic machines, 2019.
- [7] Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks, 2025.
- [8] Henry A. Kautz. The third ai summer: Aaai robert s. engelmore memorial lecture. *AI Magazine*, 43(1):105–125, 2022.
- [9] Robin Manhaeve, Sebastijan Dumancic, Angelika Kimmig, Thomas Demeester, and Luc De Raedt. Deepproblog: Neural probabilistic logic programming. In *Advances in Neural Information Processing Systems*, volume 31, 2018.
- [10] Prime Intellect. Intellect-3-rl logic environment, 2025. https://github.com/PrimeIntellect-ai/research-environments/tree/main/environments/logic_env.
- [11] Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi-Yadkori. Hierarchical reasoning model, 2025.
- [12] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2022.